Monte Carlo Proxy Pricing

A textbook-style methodology for European, American, Asian, barrier, cliquet, basket Asian, SLV cliquet, basket cliquet, autocallable, and random-payoff proxies

Proxy Pricing Research Project

July 11, 2026

Contents

Reader's Guide

This document is written as a second-year undergraduate text. It assumes calculus, linear algebra, basic probability, and a willingness to read a few stochastic-process formulas. The goal is not to hide the implementation behind jargon. The goal is to explain the finance problem, motivate the proxy solution, show the diagnostic graphs, and then derive the mathematics carefully enough that another researcher can improve the method.

The first six body chapters are intentionally practical:

- Motivation: why repeated pricing creates a computational problem.
- Solution: how the proxy workflow solves the problem.
- Graphs: how to read the value and error diagnostics.
- Examples: how the state choice changes by product.
- Feature engineering: how to turn financial state into useful proxy inputs.
- Method selection: why each proxy method exists and when it should win.

The later chapters explain the finance, the mathematics, and the implementation details. The appendices give line-by-line proofs for the formulas used in the code. Every nontrivial implementation trick is classified as one of:

- an exact pricing identity,
- an unbiased Monte Carlo variance-reduction identity,
- a numerical approximation whose error must be validated.

Notation and Conventions

The same notation is used throughout the document.

Symbol	Meaning
S_t	Spot price at time t. For baskets, $S_t^{(j)}$ is asset j.
r, q, σ	Risk-free rate, dividend yield, and volatility in GBM examples.
τ	Remaining time to maturity, usually $T - t$.
H	Final payoff or discounted cash-flow functional before discounting.
$V(t, x)$	Model value at time t when the pricing state is x.
X_t	State vector used by the proxy.
Z	Standard-normal simulation shocks after inverse-CDF mapping from Sobol points.
N	Number of Monte Carlo paths or number of fixings, depending on context. The surrounding formula identifies which one.
A_i, R_i, a_i	Running sums: Asian running sum, basket running sum, and cliquet accrued return.
ℓ, u	Local floor and cap for a cliquet coupon.
G_L, G_U	Global floor and cap for a cliquet payoff.
η	Relative-error denominator floor.

All prices in the experiments are model prices. If the simulator is GBM, SLV, or a tree, the proxy is judged against that model. A proxy can be accurate relative to the model and the model can still be a poor market model. The two questions are deliberately separated.

Chapter 1

Motivation

1.1 The practical problem

Modern risk systems often need many derivative prices, not one derivative price. A trader may ask for one price at one market state. A risk engine may need prices across thousands of simulated future states, stress scenarios, portfolio dates, and model configurations.

If one Monte Carlo price costs seconds, then a large exposure run can become too slow. The direct approach is

$$\text{market state} \quad \longrightarrow \quad \text{full pricing model} \quad \longrightarrow \quad \text{price.}$$

The proxy approach inserts an offline learning stage:

$$\text{market state} \quad \longrightarrow \quad \text{fast proxy} \quad \longrightarrow \quad \text{approximate model price.}$$

The aim is not to replace financial modeling. The aim is to preserve the chosen model's price function while making repeated evaluation cheap.

1.2 Why this is not ordinary curve fitting

Derivative prices have financial structure. A good proxy should respect as much of that structure as possible.

- A call price should not become negative.
- A vanilla call is increasing in spot.
- An American value is at least intrinsic value.
- A barrier price can change rapidly near the barrier.
- A cliquet value has exact floor and cap regions.
- A basket product may depend on level, dispersion, correlation, and order statistics.

So the project does not ask, "Which generic machine-learning method fits best?" The better question is:

> Which financial state, target transform, variance reduction, and fitted proxy work together?

1.3 Success criteria

The experiments judge a proxy by several criteria, because one number can hide too much.

- *Accuracy*: small signed, absolute, and relative errors against an independent benchmark.
- *Tail control*: acceptable error in deep in-the-money and out-of-the-money regions.
- *Stability*: no artificial oscillation near kinks, barriers, or exercise boundaries.
- *Speed*: fast evaluation after training.
- *Generality*: the method should extend from European options to path-dependent and higher-dimensional products.

The most important accuracy number in this project is often worst-case relative error with a denominator floor. The floor prevents a nearly zero benchmark from producing a meaningless huge percentage.

Chapter 2

Solution Overview

2.1 The core idea

For each product, write the expensive model price as a conditional expectation:

$$V(t,x) = \mathbb{E}_{\mathbb{Q}}[Y \mid X_t = x],$$

where Y is the discounted future payoff and X_t is the state. Training means choosing states x_i, estimating labels $V(t,x_i)$, and fitting a fast approximation $\widehat{V}(t,x)$.

The current generic methodology is:

1. Use finance to choose a pricing-sufficient state.
2. Transform the state into numerically stable coordinates.
3. Generate Monte Carlo labels with Sobol paths, antithetics, control variates, and likelihood-ratio importance sampling when useful.
4. Use exact regions whenever the payoff logic gives them.
5. Fit a shape-preserving one-dimensional proxy or a sparse higher-dimensional proxy.
6. Validate against an independent benchmark.

2.2 Workflow graph

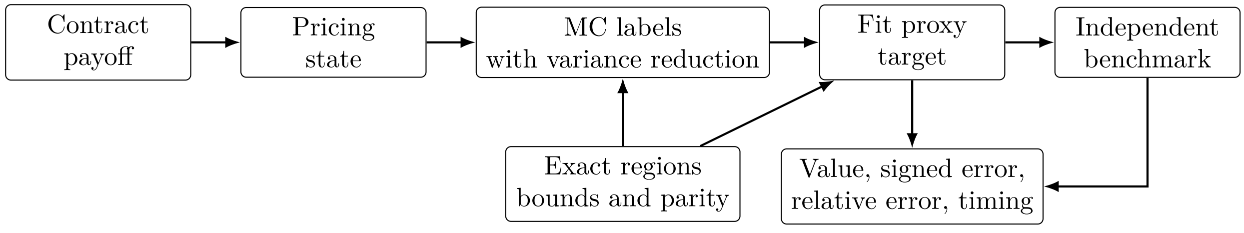

Figure 2.1: Proxy construction workflow. The fitted method is only one part of the solution; state choice, variance reduction, exact payoff regions, and independent validation are equally important.

2.3 Default method choices

For one-feature problems, the current default is SciPy PCHIP on a financially meaningful coordinate. This is used because it is local, shape-preserving, and stable near kinks. For higher-dimensional problems, the default moves toward sparse Chebyshev regression, PCA summaries, moment baselines, and residual calibration. For hard basket cliquet cases, a cached Sobol likelihood-ratio safety proxy is allowed when the fitted surface alone does not meet the error target.

Chapter 3

Diagnostic Graphs

3.1 Why graphs come before tables

Tables are useful for comparing many cases, but graphs reveal the failure mode. A method with a good average error can still fail in a wing, near a barrier, or at an exercise kink. For that reason the project uses a three-panel diagnostic:

- benchmark value and proxy value;
- signed price error, proxy minus benchmark;
- signed relative error with a denominator floor.

The sign matters. A positive signed error means the proxy overprices relative to the benchmark. A negative signed error means it underprices.

3.2 Representative validation dashboard

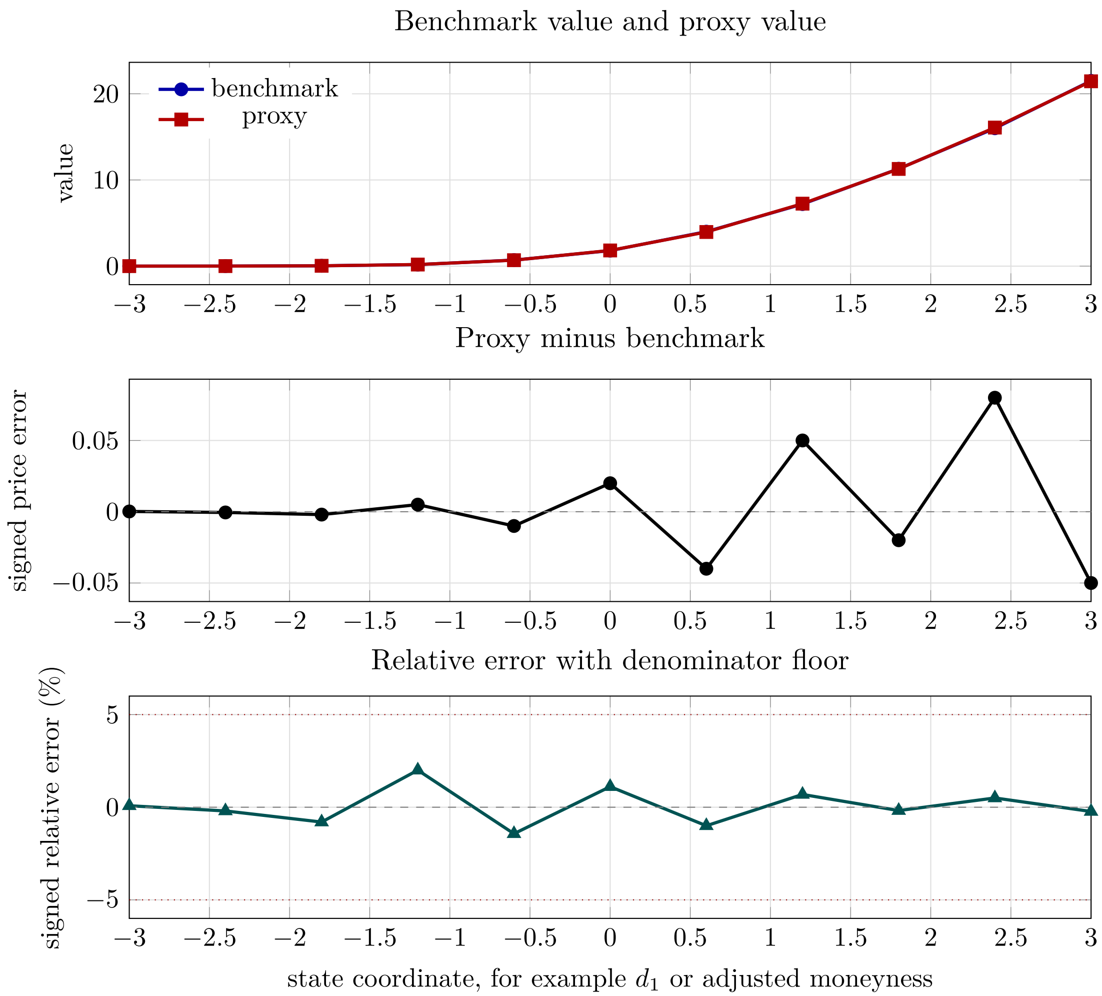

Figure 3.1: Representative diagnostic shape. The coordinates are illustrative, not a fresh benchmark run. The purpose is to show the graph format used throughout the project: values on top, signed price error in the middle, and signed relative error with a practical floor at the bottom.

3.3 How to interpret the panels

The top panel answers the first financial question: does the proxy have the same shape as the model value? The middle panel answers the pricing question: where does the proxy overprice or underprice in currency units? The bottom panel answers the risk-control question: is the percentage error acceptable after avoiding meaningless division by tiny values?

For a one-dimensional product, a good proxy usually has a smooth signed-error curve around zero. A sudden spike suggests a missing knot, a bad training label, a barrier/exercise transition, or an extrapolation problem. For a higher-dimensional product, the same idea is applied to slices, stress paths, and validation grids.

Chapter 4

Guided Examples

4.1 A European call proxy

Example 4.1 (One state variable is enough)**.** Consider a one-year European call with $S = 100$, $K = 100$, $r = 5\%$, $q = 2\%$, and $\sigma = 20\%$. Under GBM, once today's spot is known, the future distribution of S_T does not depend on the earlier path. Thus the pricing state is just $X_t = S_t$.

The proxy should not fit raw spot blindly. A better coordinate is

$$d_1 = \frac{\log(S/K) + (r - q + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}.$$

This coordinate is financially meaningful because Black-Scholes delta is $\Phi(d_1)$ for a non-dividend-adjusted call. Spacing points in d_1 gives more balanced coverage of option-delta regions than spacing points uniformly in spot.

4.2 An Asian option running-sum example

Example 4.2 (Why the running sum matters)**.** Suppose an arithmetic Asian call has 12 monthly fixings and strike $K = 100$. Four fixings have already occurred, with running sum

$$A = 98 + 101 + 103 + 99 = 401.$$

Today is a fixing date and today's spot is $S = 105$. There are $m = 7$ future fixings after today. The final average is

$$\bar{S}_T = \frac{401 + 105 + \sum_{j=1}^{7} S_{t_j}}{12}.$$

The payoff $(\bar{S}_T - 100)_+$ is equivalent to a scaled option on the future average growth factors. The adjusted strike is

$$K_{\text{adj}} = \frac{12 \cdot 100 - 401 - 105}{7} = 99.1429.$$

The relevant moneyness is therefore $K_{\text{adj}}/S = 99.1429/105 = 0.9442$. This example explains why the proxy uses a coordinate involving both S and A, not spot alone.

4.3 A barrier example

Example 4.3 (Same spot, different barrier state)**.** Consider a down-and-out call with lower barrier $L = 80$. Two paths can both have today's spot $S_t = 100$. If the first path never touched 80, the option is alive. If the second path touched 80 earlier, the option has already knocked out and is worth zero. The same spot therefore has two different values.

The state must include an alive indicator I_t. A spot-only proxy would average alive and dead cases together, which is financially wrong. This is an example where adding one discrete state variable fixes a serious missing-information problem.

4.4 An American put example

Example 4.4 (Exercise versus continuation)**.** Suppose an American put has strike $K = 100$ and today's spot is $S = 70$. Immediate exercise pays $K - S = 30$. If the discounted expected continuation value is only 28, the value is 30. If the continuation value is 33, the value is 33.

The recursion is

$$V(t, S) = \max\{K - S,\ C(t, S)\},$$

where $C(t, S)$ is continuation value. A proxy should fit continuation smoothly but impose the max exactly. If it does not impose the max, it can produce values below intrinsic value, which is an arbitrage-style numerical error.

4.5 A cliquet exact-tail example

Example 4.5 (No regression needed in a global cap tail)**.** Suppose a cliquet has global cap $G_U = 20\%$, local floor $\ell = -2\%$, local cap $u = 4\%$, current accrued return $a = 18\%$, and five coupons remaining. Even if every future coupon is as low as possible, the final accumulated return is at least

$$18\% + 5(-2\%) = 8\%.$$

So the global cap is not guaranteed. But if $a = 24\%$, then even before future coupons the final clipped payoff is already capped at 20%, so the value is exactly the discounted capped payoff.

Exact regions like this are important. They reduce MC noise and stop the fitted proxy from learning a flat floor or cap region from noisy labels.

4.6 A basket Asian example

Example 4.6 (Same basket level, different composition)**.** Consider a two-asset equal-weight basket. State A has spots $(100, 100)$. State B has spots $(130, 70)$. Both have basket level 100. If the assets have different volatilities or dividends, or if the payoff depends on nonlinear future averages, the two states need not have the same value.

This explains why a basket proxy often needs composition features, not only basket level. PCA features are a compact way to describe composition and dispersion when the number of underlyings is large.

4.7 A basket cliquet order-statistic example

Example 4.7 (Worst-of identity can switch)**.** Suppose a three-asset worst-of cliquet coupon uses

$$\min(R^{(1)}, R^{(2)}, R^{(3)}).$$

At one state, asset 1 may be most likely to be worst. At a nearby state, asset 2 may become the likely worst asset. The coupon surface can bend sharply near the switching region.

This is why basket cliquet was harder than European or Asian options. The proxy must handle not only level and volatility but also cross-sectional ranking behavior.

Chapter 5

Feature Engineering for Proxy Pricing

5.1 Why feature engineering matters

For proxy pricing, feature engineering is often more important than the choice between PCHIP, Chebyshev, Akima, or a neural network. A fitting method can only learn a function of the inputs it receives. If the input omits a state variable that changes the future payoff distribution, then the approximation error is structural. More paths and a more powerful regressor cannot fully repair it.

The guiding principle is:

first make the state financially sufficient, then make it numerically easy.

A financially sufficient state contains the information needed to price future cash flows. A numerically easy feature vector rescales and reshapes that state so that the fitted proxy sees a smoother function.

5.2 State variables versus features

It is useful to separate two concepts:

- the *state* is the information needed for pricing;
- the *feature vector* is the numerical representation given to the fitting method.

For a European option under GBM, the state can be S_t, but the feature can be d_1. For an Asian option, the state can be (S_t, A_t), but the feature can be adjusted moneyness K_{adj}/S_t. For a basket Asian, the state contains all spots and the running basket sum, but the feature vector may contain basket level, running-sum cushion, PCA scores, and a moment baseline.

5.3 Feature engineering workflow

The recommended workflow is:

1. Write the payoff and observation schedule.
2. Identify the Markov or pricing-sufficient state.
3. Add discrete status variables such as alive, called, knocked out, or exercise date.
4. Find exact regions and bounds before fitting.
5. Build dimensionless coordinates such as moneyness, log-moneyness, d_1, accrued cushion, or normalized variance.
6. Add baselines, moments, and residual targets when the raw value is too curved.
7. Add cross-sectional features for baskets: level, dispersion, PCA, best, worst, and spread indicators.
8. Validate feature sufficiency by plotting residuals against omitted candidate variables.

5.4 Common feature types

Feature type	Examples	Rationale
Level	S/K, basket level, forward moneyness	Captures the main price direction.
Log level	$\log(S/K)$, log spot vector	GBM evolves additively in log space.
Greek-inspired	d_1, delta bucket, vega scale	Places grid resolution where option sensitivity changes.
Path state	running sum, accrued coupon, realized average	Restores sufficiency for path-dependent payoffs.
Status flag	alive, called, knocked out, exercise date	Separates states with different payoff rules.
Bound/cushion	distance to floor, cap, barrier, strike	Makes exact regions and transition regions visible.
Moment baseline	conditional mean, variance, lognormal baseline	Removes the dominant smooth component before fitting.
Cross-sectional	basket level, dispersion, best, worst, spread	Captures basket composition and ranking effects.
Dimension reduction	PCA scores, common factor, residual factors	Avoids full tensor grids in high dimension.
Model state	variance v_t, local-vol bucket, rate state	Includes variables that affect future dynamics.

5.5 One-feature examples

5.5.1 European option

Raw spot is a valid state, but d_1 is usually a better feature:

$$d_1 = \frac{\log(S/K) + (r - q + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}.$$

The reason is not cosmetic. The Black-Scholes call delta is tied to $\Phi(d_1)$, so equal spacing in d_1 gives more balanced coverage across delta regions than equal spacing in spot.

5.5.2 Asian option

For an arithmetic Asian call with running sum A, current fixing S, and m future fixings, the adjusted strike is

$$K_{\text{adj}} = \frac{NK - A - S}{m}.$$

The important feature is not S alone. It is the adjusted moneyness

$$\frac{K_{\text{adj}}}{S}.$$

This feature combines today's spot and the running sum into the object that the remaining future average must beat.

5.5.3 Barrier option

For a barrier option, distance to the barrier is more informative than raw spot:

$$b_{\text{dist}} = \log(S/L)$$

for a lower barrier L. The state also needs the alive flag. Without the alive flag, two paths with the same spot can have different values.

5.5.4 American and Bermudan options

For early-exercise products, the feature should expose both moneyness and time to remaining exercise opportunities. A continuation proxy can use spot or log-moneyness, but the exercise rule must impose

$$V = \max(\text{intrinsic}, \text{continuation})$$

after fitting. A feature that hides the exercise boundary tends to create oscillatory errors.

5.6 Basket and high-dimensional examples

5.6.1 Basket Asian

A basket Asian needs more than basket level. Two states can have the same basket value but different composition. Useful features include:

- current basket level;
- running basket sum or remaining average cushion;
- weighted log spots;
- cross-sectional dispersion;
- PCA scores of centered log spots;
- conditional mean and variance of the future basket average;
- lognormal moment baseline and fitted residual.

The moment baseline handles much of the smooth level effect. PCA and dispersion features explain composition effects that a scalar basket level cannot see.

5.6.2 Basket cliquet

Basket cliquets need features that describe ranking and dispersion, not only level. For a worst-of or best-of coupon, the identity of the best or worst asset can switch. Useful features include:

- accrued coupon and remaining floor/cap cushion;
- basket return level;
- minimum and maximum recent return indicators;
- cross-sectional spread;
- PCA scores for log spots and log variances;
- variance state for each underlying;
- local-volatility or SLV state summaries.

If the switching surface is too sharp, a fitted smooth proxy may need a cached Sobol/LR safety layer in that region.

5.7 Feature transforms and target transforms

Feature engineering is linked to target engineering. Some examples:

- Use $\log(V + \epsilon)$ for positive prices with wide dynamic range.
- Use a bounded logit target for known floor/cap products.
- Use $\log((V + \epsilon)/(B + \epsilon))$ when a baseline B captures most of the shape.
- Use signed or additive residuals when the baseline can be close to zero.

The feature and target should be chosen together. A good feature can make the target nearly linear or nearly flat; a poor feature can make even a flexible regressor work too hard.

5.8 How to detect missing features

After fitting, do not only look at the overall error table. Plot signed errors against candidate omitted variables. Warning signs include:

- residuals that trend with running sum after using spot only;
- residuals that trend with distance to barrier;
- basket residuals that trend with dispersion or worst-minus-best spread;
- cliquet residuals that trend with accrued floor/cap cushion;
- SLV residuals that trend with variance after using spot only.

If residuals have structure against an omitted variable, the problem is probably feature engineering, not interpolation.

5.9 Feature selection without becoming ad hoc

The feature set should be product-aware but not arbitrary. A defensible feature usually comes from one of four sources:

1. state sufficiency: the variable is needed by the payoff or dynamics;
2. asymptotics: the variable controls a known tail, bound, or exact region;
3. moment approximation: the variable enters conditional mean, variance, or a baseline;
4. validation residuals: independent errors show a stable pattern against the variable.

This keeps the methodology generic. It allows product-specific features, but requires each feature to have a mathematical, financial, or empirical reason.

5.10 Practical feature checklist

Before training a proxy, check:

- Does the state determine future payoff distribution under the model?
- Are all path-dependent running quantities included?
- Are barrier, call, exercise, and knock-out statuses included?
- Are features dimensionless and scaled to comparable ranges?
- Are known exact regions removed or encoded?
- Is there a baseline that can explain the main level effect?
- For baskets, are level, dispersion, and ranking effects represented?
- For SLV, are spot and variance both represented?
- Do residual plots show structure against omitted variables?
- Does the feature design still work across parameter cases, not only one hand-picked option?

Chapter 6

Method Motivation and Selection

6.1 How to decide that one method is better

A proxy method is not better in the abstract. It is better for a particular error mode, product shape, label quality, and dimension. The project therefore compares methods using several criteria:

Criterion	Meaning
Worst relative error	Controls the bad validation point and matters for production sign-off.
p99 relative error	Focuses on tail quality without letting one pathological point dominate.
Signed error smoothness	Reveals missing knots, Monte Carlo noise, extrapolation artifacts, and local bias.
Shape safety	Avoids negative prices, artificial extrema, and exercise violations.
Training cost	Time to generate labels and fit the proxy.
Pricing speed	Time to evaluate the proxy after training.
Generality	Whether the same idea transfers to the next product family.

For PFE and exposure work, shape safety can matter as much as a small average accuracy improvement. A method that reduces average error but creates a fake local bump can still be dangerous because exposure quantiles depend on tails and local maxima.

6.2 Decision map

The current practical map is:

Situation	First method to try	Reason

One feature, clean labels, kinks possible	PCHIP	Local, shape-preserving, no artificial extrema.
One feature, many slope changes	Akima or MAKIMA	Local cubic slopes react smoothly to uneven local behavior.
One feature, very smooth global surface	Chebyshev or Bernstein ridge	Regularized global basis denoises MC labels.
One feature, noisy labels	Smoothing spline or P-spline	Trades exact interpolation for lower variance.
Early-exercise continuation	PCHIP continuation plus exact max	Avoids oscillatory exercise boundary errors.
Known floor or cap	Bounded target plus PCHIP or Chebyshev	Enforces financial bounds by construction.
Smooth multi-feature proxy	Sparse Chebyshev ridge	Fast, stable, interpretable, convex fit.
Correlated basket proxy	Baseline plus PCA sparse correction	Captures level and composition without a full tensor grid.
Order-statistic basket cliquet	Cached Sobol/LR safety proxy	Switching surfaces are too hard for one smooth fitted surface.

6.3 Linear interpolation

Linear interpolation is the simplest local baseline. It connects neighboring labels by straight lines.

It can be better when labels are noisy, when no artificial curvature is acceptable, or when an audit needs a transparent benchmark. It can be worse because it is only first-order accurate on smooth curves and can require a dense grid to match cubic accuracy.

Its main role is diagnostic. If a complicated method performs much worse than linear interpolation, the complicated method is probably unstable for that product.

6.4 PCHIP

PCHIP is a local piecewise cubic Hermite interpolant with slope limiting. Its motivation is financial shape safety: option prices often have monotone regions, flat tails, exercise kinks, and barrier transitions.

PCHIP can be better because it is local, fast, shape-preserving on monotone data, and resistant to artificial extrema. In the expanded one-feature study it had zero local overshoot. It can be worse because it interpolates MC labels exactly. If labels are visibly noisy, PCHIP preserves that roughness instead of denoising it.

PCHIP remains the generic one-feature default because it is the best compromise between accuracy, locality, shape safety, speed, and explainability.

6.5 Akima and MAKIMA

Akima methods also build local cubic interpolants, but their slopes are weighted from nearby secants rather than limited primarily for monotonicity. The motivation is to handle local slope changes smoothly.

They can be better when the target has many local slope changes but does not require strict monotonicity. This is why Akima was strong in the Asian adjusted-moneyness and random-payoff tests. They can be worse because they are less strictly shape-preserving than PCHIP and can introduce small overshoot.

Akima is the first local challenger when PCHIP looks too conservative or too angular.

6.6 Natural cubic interpolation

Natural cubic interpolation passes through all labels and gives a twice continuously differentiable curve with zero second derivative at the endpoints.

It can be better when the true value surface is smooth and labels are clean. It was fractionally best on the product-balanced p99 metric in the expanded one-feature study. It can be worse because it is globally coupled and can overshoot between labels. That is a serious drawback near barriers, exercise boundaries, and payoff tails.

The conclusion is that natural cubic interpolation is a credible accuracy challenger, but not the default for risk use because its small accuracy edge did not compensate for overshoot risk.

6.7 B-spline regression

B-spline regression represents a curve using local polynomial basis functions. It is the stable version of regression splines and truncated-power splines.

It can be better when MC labels are noisy and exact interpolation is too literal. It can place flexibility near important regions through knot choice. It can be worse because knot placement and regularization become design choices. If knots miss a barrier or exercise feature, the method can blur the feature.

B-spline regression is a good denoising candidate, not the simplest production default.

6.8 Smoothing splines and P-splines

A smoothing spline balances label fit and roughness:

$$\sum_i (y_i - f(x_i))^2 + \lambda \int (f''(x))^2 \, \mathrm{d}x.$$

A P-spline uses a B-spline basis with a penalty on coefficient differences. Both methods are motivated by MC noise reduction.

They can be better when label variance is the main problem. They can be worse when the smoothing penalty blurs a true kink, barrier transition, or exercise boundary. Generic smoothing parameters can also underfit steep log-price tails.

The practical use is: try them after improving labels with Sobol, antithetics, control variates, and importance sampling. They should not be used to hide a bad simulator.

6.9 Adaptive P-splines and local regression

Adaptive P-splines allow different smoothness in different regions. LOESS fits local low-degree polynomials around each prediction point. Both are motivated by varying local behavior.

They can be better near curves with smooth tails and sharp transition regions. They can be worse because they introduce more hyperparameters and are harder to audit. LOESS can also be slow at pricing time unless carefully cached.

These methods are useful research tools when ordinary PCHIP or fixed-smoothing splines leave visible structured errors.

6.10 Gaussian-process smoothers

Gaussian-process smoothers treat the unknown price function as a random smooth function with a covariance kernel.

They can be better because they provide uncertainty estimates and can guide adaptive state sampling. They can be worse because training scales poorly with many states, kernel choices matter, and pricing is slower than interpolation or linear regression.

Their best role in this project is experimental design: use uncertainty to choose where to simulate more labels.

6.11 Chebyshev ridge

Chebyshev polynomials are well conditioned on $[-1, 1]$, and ridge regularization stabilizes noisy labels. This makes Chebyshev ridge attractive for smooth value surfaces.

It can be better because it is fast, convex, stable after scaling, and able to denoise through regularization. It can be worse near kinks, barriers, and switching surfaces because it is a global basis. High degree can also overfit MC noise.

For smooth European-style surfaces and low-dimensional smooth corrections, Chebyshev ridge is often the strongest global method.

6.12 Piecewise Chebyshev

Piecewise Chebyshev tries to keep spectral accuracy while reducing the global oscillation problem.

It can be better when different regions of the state space have different smoothness. It can be worse because the breakpoints, overlaps, and continuity rules become new modeling choices.

It is a promising bridge between local PCHIP and global Chebyshev, especially for products with smooth pieces separated by transition regions.

6.13 Bernstein and Bezier methods

Bernstein polynomials have useful shape properties on bounded intervals. Bezier curves are an equivalent geometric representation of polynomial segments.

They can be better when positivity, monotonicity, or convexity constraints are imposed through coefficients. They can be worse when used unconstrained, because they are still global fits. A piecewise Bezier curve using PCHIP slopes is not a new method; it is PCHIP written in Bezier form.

Their best future use is constrained regression when the finance gives a known shape.

6.14 Floater-Hormann rational interpolation

Floater-Hormann interpolation is a pole-free barycentric rational method. It is motivated by the fact that rational functions can approximate some smooth curves better than polynomials.

It can be better on difficult smooth approximation problems. It can be worse for option proxies because it does not automatically respect price bounds or shape constraints. In the expanded study it produced unacceptable excursions on some barrier log-value tests.

It is rejected as a generic default.

6.15 Sparse Chebyshev for multiple features

In more than one feature, the full tensor polynomial basis grows too quickly. Sparse Chebyshev ridge keeps low-order and payoff-relevant interactions.

It can be better because the fit is convex, fast at pricing time, interpretable, and compatible with analytic derivatives. It can be worse because the basis terms must be selected carefully. Missing an important interaction creates bias that more paths will not remove.

The rule is: use sparse Chebyshev only after scaling features and thinking carefully about which interactions the payoff can create.

6.16 Baseline plus residual

A baseline plus residual proxy uses finance to explain the main price shape and fits only the correction:

$$V(x) = B(x)R(x) \qquad \text{or} \qquad V(x) = B(x) + \varepsilon_{\text{res}}(x).$$

It can be better because the residual is flatter and lower dimensional than the raw value. It can be worse if the baseline is poor or nearly zero in an important region. The ratio target must therefore include a floor.

This is one of the most important ideas for higher-dimensional products: feature design and baseline design usually matter more than the name of the regression method.

6.17 PCA features

PCA is motivated by basket structure. A basket can have many spots, but much of the variation is common level plus a few dispersion directions.

PCA can be better because it reduces dimension while preserving dominant cross-sectional variation. It can be worse because it is linear and unsupervised. For order-statistic payoffs, the most important direction may be an extreme or ranking direction, not the largest-variance direction.

PCA should therefore be used together with payoff-aware features such as basket level, accrued return, volatility state, and floor/cap cushion.

6.18 Neural networks

Neural networks are flexible high-dimensional function approximators. They are motivated by products whose interactions are too complex for a small sparse polynomial basis.

They can be better in genuinely high dimension, especially with large training sets. They can be worse because they are harder to audit, need architecture and training choices, and do not automatically respect financial constraints.

They are a future benchmark for high-dimensional exotics, not the current default.

6.19 Cached Sobol/LR safety proxy

Some products are too hard for one fitted smooth surface. Basket cliquet order-statistic coupons can switch which asset is best or worst. A cached Sobol likelihood-ratio safety proxy prices difficult states by reusing a strong simulator rather than relying on a weak fitted extrapolation.

It can be better because it is much safer in discontinuous or switching regions. It can be worse because it is slower than an analytic fitted proxy and still has MC error.

This is the current safety fallback for basket cliquet hard cases.

6.20 GPU acceleration

GPU acceleration is not a new pricing theory and not a new proxy family. It is a computational backend choice. The important question is where the parallel work actually lives.

The strongest GPU candidate is Monte Carlo label generation:

$$x_i \longmapsto \frac{1}{N}\sum_{n=1}^{N} e^{-r\tau} H(x_i, Z_n).$$

For each state x_i, many paths can be simulated independently. For each path, many assets, time steps, payoff variants, and likelihood-ratio weights can be computed in parallel. This is exactly the kind of work GPUs are good at.

The same argument applies to independent benchmark generation. If a benchmark requires hundreds of thousands or millions of paths at many validation states, then the benchmark is often more GPU-friendly than the final fitted proxy.

Task	GPU usefulness	Rationale
MC training labels	High	Paths, states, assets, and payoff variants are massively parallel.
MC benchmarks	High	Same structure as training labels, often with even more paths.
Basket Asian simulation	High	Many assets, correlations, and state points.
SLV and basket cliquet simulation	High	Many time steps, variance states, coupons, and LR weights.
Barrier simulation	Medium to high	Path monitoring and Brownian-bridge factors are parallel, but branch behavior near barriers needs care.
PCHIP, Akima, linear interpolation	Low	Sorting and slope construction are small CPU tasks.
Small Chebyshev ridge fits	Low to medium	CPU is usually enough unless the design matrix is large or many fits are batched.
Large sparse basis evaluation	Medium	GPU can help build A, $A^\top A$, and batched predictions.
Neural-network training	High	Matrix operations and minibatch training are GPU-native.

The current practical recommendation is therefore not "move everything to the GPU." It is to use the GPU for labels and heavy high-dimensional training, while keeping small deterministic fitting and reporting on CPU.

There is one important caveat. This project prefers Sobol and other low-discrepancy designs for MC labels. GPU pseudo-random normals are easy, but replacing Sobol with pseudo-random paths just to use the GPU could increase variance. A careful GPU plan should test one of the following:

- generate Sobol points on CPU and transfer them to GPU in chunks;

- use a GPU library with reliable Sobol or quasi-Monte Carlo support;
- compare GPU pseudo-random antithetic labels against CPU Sobol labels at equal error, not merely equal path count.

Likelihood-ratio mixtures also need numerical care on GPU. It is safer to compute likelihood weights in log space and normalize only when necessary, because tail shifts can create very small or very large weights.

The proposed architecture is a backend abstraction:

$$\text{simulate\_labels}(\text{contract}, \text{states}, \text{config}, \text{backend}).$$

The backend choices could include CPU Sobol, GPU Sobol, and GPU pseudo-random antithetic modes. The validation report should compare wall time, standard error, and final proxy error for each backend.

This is still open for discussion. GPU acceleration is most attractive if the training or benchmark time is dominated by paths. It is less attractive if the run is dominated by Python overhead, file output, small spline fits, or model logic with heavy branching.

6.21 Recommended method ladder

Before inventing a new method, use this sequence:

1. Verify that the state is pricing-sufficient.
2. Add exact payoff regions: maturity, intrinsic value, barrier hit state, floor/cap tails, and in/out parity.
3. Improve labels using Sobol paths, antithetics, control variates, common random numbers, and LR mixture importance sampling.
4. Choose a financially scaled coordinate such as d_1, adjusted moneyness, accrued cushion, normalized variance, or PCA score.
5. Try PCHIP for one feature and sparse Chebyshev for several smooth features.
6. If labels are noisy, test smoothing spline or P-spline.
7. If local slope changes dominate, test Akima or MAKIMA.
8. If the target is globally smooth, test Chebyshev or Bernstein ridge.
9. If the product is high-dimensional, add baseline plus residual and PCA.
10. If strict max error still fails, use adaptive sampling or a cached Sobol/LR safety proxy in the hard region.
11. If label or benchmark generation dominates runtime, test a GPU MC backend while preserving Sobol/LR accuracy controls.

6.22 Further improvements

The most valuable next improvements are:

- adaptive state sampling driven by validation error;
- state-dependent MC path allocation;
- shape-constrained splines or Bernstein fits where monotonicity, convexity, or bounds are known;
- adaptive sparse Chebyshev basis selection;
- product-aware LR mixture components;
- GPU backends for MC label generation, benchmarks, and neural-network training;
- outer-scenario weighted validation for PFE;
- neural-network benchmarks for genuinely high-dimensional products.

Chapter 7

Financial Background

7.1 What a derivative price means

A derivative is a contract whose cash flows are determined by market variables such as spot prices, interest rates, volatilities, realized averages, barrier events, or coupon-reset rules. A European call, for example, pays $(S_T - K)_+$ at maturity. An Asian option pays a function of an average. A cliquet pays a function of capped and floored period returns.

The mathematical payoff is not yet the price. A price is the amount of money at the valuation date that is economically equivalent to the future cash flows under the chosen pricing model. In a complete frictionless model, this equivalence is understood through replication: if a self-financing trading strategy produces the same future cash flow as the derivative, then the derivative must have the same initial cost as that strategy. Otherwise an arbitrage would exist.

For this project, the word *price* always means *model price*. The model may be GBM, SLV, a tree, a finite-difference method, or a Monte Carlo simulator. The proxy is judged by how well it approximates the model price. It does not by itself prove that the model is a good description of the market.

7.2 Why risk-neutral pricing appears

It is tempting to think that a derivative price is the discounted real-world expected payoff,

$$e^{-r(T-t)}\mathbb{E}_{\mathbb{P}}[H \mid \mathcal{F}_t],$$

where $\mathbb{P}$ is the real-world probability measure. That is usually wrong. Risky cash flows earn risk premia, and the physical drift of the underlying asset is not the right drift for no-arbitrage valuation.

Risk-neutral pricing changes the probability measure from $\mathbb{P}$ to $\mathbb{Q}$ so that discounted tradable asset prices are martingales. In the Black-Scholes model, this means the stock drift becomes $r - q$, not the investor's expected return μ. The pricing formula then becomes

$$V_t = B_t\,\mathbb{E}_{\mathbb{Q}}\left[\frac{H}{B_T}\,\middle|\,\mathcal{F}_t\right].$$

This is not saying investors are risk-neutral. It is saying that, after risk has been priced through the change of measure, the derivative can be valued as a discounted expectation under $\mathbb{Q}$.

Remark 7.1 (Model choice and market calibration)**.** In practice, desks calibrate models to liquid market instruments before using them for less liquid instruments. The proxy work in this repository starts after that modeling choice has been made. The central question is: given a pricing model, can we replace repeated expensive valuation with a fast and accurate approximation?

7.3 Why proxies are useful

Monte Carlo pricing can be expensive because one price may require many simulated paths. A risk report, PFE calculation, XVA run, stress test, or calibration loop may need thousands or millions of prices. A proxy replaces the expensive pricing function

$$x \longmapsto V(t, x)$$

with a cheaper function

$$x \longmapsto \widehat{V}(t, x).$$

The proxy is trained offline using expensive labels and then used online for fast repeated evaluation.

This separation is the main engineering tradeoff:

$$\text{spend time once to train} \quad \Longrightarrow \quad \text{save time many times later.}$$

The proxy is valuable only if it preserves the financial shape of the price: monotonicity, convexity when relevant, exercise kinks, barrier transitions, floor/cap regions, and tail behavior.

7.4 Product families and path dependence

The products in this project differ mainly in what information from the past is needed to value the future.

Product	Financial feature	State idea
European	payoff depends only on terminal spot	current spot is enough under GBM
Asian	payoff depends on an average	current spot plus running sum
Basket Asian	payoff depends on an average basket	all current spots plus running basket sum
American/Bermudan	holder may exercise early	current spot and exercise date
Barrier	payoff depends on whether a level was hit	current spot plus alive/hit status
Cliquet	payoff depends on capped/floored reset returns	accrued coupon plus market state

Autocallable	early redemption at observation dates	spot, date, and alive status

This is why proxy design is not only a regression problem. It is first a financial-state problem. If the state omits information that affects future cash flows, then no interpolation method can fully repair the error.

7.5 Model risk, numerical risk, and proxy risk

There are three different errors that should not be confused.

- *Model risk*: the chosen dynamics may not match the market.
- *Numerical risk*: the benchmark simulator may have time-step, path-count, or discretization error.
- *Proxy risk*: the fitted approximation may differ from the benchmark model value.

This repository mainly studies proxy risk and numerical risk under specified models. When closed-form benchmarks exist, as for European Black-Scholes options, they are used. When they do not exist, the benchmark is a stronger numerical calculation, usually with more paths, independent randomization, and variance reduction.

Chapter 8

Mathematical Background

8.1 Conditional expectation as the pricing target

The central mathematical object is a conditional expectation:

$$V(t,x) = \mathbb{E}_{\mathbb{Q}}[Y \mid X_t = x],$$

where Y is the discounted future payoff and X_t is the state. This formula says: among all scenarios that share the same current state x, average the discounted future payoff under the pricing measure.

For a second-year undergraduate, the useful intuition is that conditional expectation is the best mean-square prediction using the information currently available. If $g(X_t)$ is any function of the state, then

$$\mathbb{E}\big[(Y - \mathbb{E}[Y \mid X_t])^2\big] \leq \mathbb{E}\big[(Y - g(X_t))^2\big].$$

So proxy pricing can be viewed as approximating the best possible state-based prediction of the discounted payoff.

8.2 Pricing-sufficient states

Definition 8.1 (Pricing-sufficient state)**.** Let $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t\geq 0}, \mathbb{Q})$ be a filtered risk-neutral probability space, and let H be an integrable payoff. An adapted process $X_t \in \mathbb{R}^d$ is called pricing-sufficient for H at time t if there exists a measurable function v such that

$$B_t \, \mathbb{E}_{\mathbb{Q}}\left[\frac{H}{B_T} \,\middle|\, \mathcal{F}_t\right] = v(t, X_t).$$

This definition is the professional version of the earlier informal phrase "a good state is sufficient." It says exactly what must be true: after conditioning on X_t, no missing history can change the price. The state may still be too high-dimensional for convenient fitting, but it is at least financially correct.

Remark 8.1 (Sufficient does not mean minimal)**.** A sufficient state may contain redundant variables. For example, including both spot and log spot is unnecessary but still sufficient. A minimal state removes redundancy, but finding the truly minimal state is often less important than finding a stable low-dimensional state that prices accurately.

8.3 Markov models and state evolution

A process is Markov if the conditional law of the future depends on the past only through the current state. In a one-asset GBM model,

$$\frac{\mathrm{d}S_t}{S_t} = (r - q)\,\mathrm{d}t + \sigma\,\mathrm{d}W_t,$$

the future distribution of S_T depends on the current history only through S_t. Thus S_t is a pricing-sufficient state for European options.

For path-dependent products, the spot alone is usually not Markov for the payoff. An Asian option also needs the running sum. A barrier option needs to know whether the barrier has already been hit. A cliquet needs the accrued coupon. The mathematical task is to enlarge the state just enough to restore the Markov property for the future cash flows.

8.4 Brownian motion, GBM, and lognormality

GBM is used throughout the simpler examples because it has a closed-form transition:

$$S_{t+\Delta t} = S_t \exp\Big((r - q - \tfrac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\,Z\Big), \qquad Z \sim N(0, 1).$$

This formula explains why log-moneyness and d_1-type variables are natural coordinates. The randomness enters the log price additively, so a coordinate based on $\log(S/K)$ is better scaled than raw spot.

For multiple assets, the normal shocks are correlated. If ρ is the correlation matrix and $LL^\top = \rho$, then LZ has correlation ρ when Z has independent standard-normal components. This is the linear algebra behind basket simulations.

8.5 Dynamic programming for exercise and callable products

Early-exercise and callable products are valued backward in time. At a decision date, the value is the better of stopping and continuing. For an American-style product,

$$V_k(x) = \max\{g_k(x), C_k(x)\},$$

where $g_k(x)$ is immediate exercise value and

$$C_k(x) = e^{-r\Delta t}\mathbb{E}_{\mathbb{Q}}[V_{k+1}(X_{k+1}) \mid X_k = x]$$

is continuation value.

The proxy problem is therefore not only terminal payoff fitting. It can also be continuation-value fitting. This is why monotone and shape-preserving one-dimensional methods are useful: exercise boundaries create kinks, and a global polynomial can oscillate near those kinks.

8.6 Approximation viewpoint

After MC labels are generated, fitting is an approximation problem. There are three common viewpoints.

- *Interpolation*: pass exactly through relatively clean labels. PCHIP is in this category.
- *Regression*: fit noisy labels with a lower-dimensional function class. Ridge Chebyshev regression is in this category.
- *Baseline plus correction*: use finance to explain most of the price, then fit the smaller residual.

The best method depends on dimension and noise. In one feature, shape-preserving interpolation is hard to beat when labels are accurate. In many features, sparse regression, PCA summaries, and baseline corrections become more important because grids become too large.

8.7 Curse of dimensionality

If one dimension uses m grid points, a full tensor grid in d dimensions uses m^d points. With $m = 20$ and $d = 10$, this is 20^{10}, which is impossible for routine training. This is the curse of dimensionality.

The project responds to this in several ways:

- reduce the state using finance, such as adjusted moneyness for Asian options;
- use PCA to summarize correlated cross-sectional variation;
- use sparse Chebyshev bases rather than full tensor bases;
- use baselines so that the fitted residual is simpler than the original price;
- use cached MC safety pricing for hard basket cliquet cases where a fitted proxy alone is not reliable enough.

Chapter 9

The Pricing Problem

9.1 Risk-neutral value

Let H be the payoff of a derivative at maturity or final settlement time T. Under the risk-neutral measure $\mathbb{Q}$, the time-t value is

$$V_t = B_t \, \mathbb{E}_{\mathbb{Q}}\left[\frac{H}{B_T} \,\middle|\, \mathcal{F}_t \right],$$

where $B_t = \exp(\int_0^t r_s \, \mathrm{d}s)$ is the money-market account. If the short rate is constant,

$$V_t = e^{-r(T-t)} \mathbb{E}_{\mathbb{Q}}[H \mid \mathcal{F}_t].$$

In the code, the expensive object is the conditional expectation. The proxy is a fast approximation of the map

$$x \longmapsto V(t, x),$$

where x is the state available at the valuation date.

9.2 What makes a good proxy state

A state is good when it is sufficient, low-dimensional, and scaled sensibly.

Definition 9.1 (Sufficient state)**.** A vector X_t is sufficient for pricing if, after conditioning on X_t, the distribution of all future cash flows no longer depends on earlier path history.

Examples:

- European option: $X_t = S_t$.
- Arithmetic Asian option: $X_t = (S_t, A_t)$, where A_t is the running sum of past fixings.
- American put: $X_t = S_t$, but the time index matters because of remaining exercise dates.
- Barrier option: $X_t = (S_t, I_t)$, where I_t says whether the barrier has already been hit.
- Single-name SLV cliquet: $X_t = (a_t, S_t, v_t)$, accrued coupon, spot, and variance.
- Basket SLV cliquet: $X_t = (a_t, S_t^{(1:3)}, v_t^{(1:3)})$, plus contract variant.

9.3 Error decomposition

The observed proxy error is not one number with one cause. A useful decomposition is

$$\widehat{V}_{\text{proxy}} - V_{\text{true}} = \underbrace{\widehat{V}_{\text{proxy}} - V_{\text{MC-label}}}_{\text{fitting error}} + \underbrace{V_{\text{MC-label}} - V_{\text{model}}}_{\text{training MC noise}} + \underbrace{V_{\text{model}} - V_{\text{true}}}_{\text{model/discretization error}} .$$

The experiments in this repository mostly study the first two terms under fixed model assumptions.

Chapter 10

Monte Carlo Label Construction

10.1 Plain estimator

For a state x, define the discounted path value

$$Y(x, Z) = e^{-r\tau} H(x, Z),$$

where Z represents all future random shocks. The target label is

$$V(x) = \mathbb{E}[Y(x, Z)].$$

With N simulated paths,

$$\widehat{V}_N(x) = \frac{1}{N} \sum_{n=1}^{N} Y(x, Z_n).$$

The estimator is unbiased when the paths have the correct law. Its standard error is estimated by

$$\widehat{\mathrm{SE}}(\widehat{V}_N) = \sqrt{\frac{1}{N(N-1)} \sum_{n=1}^{N} \big(Y_n - \overline{Y}\big)^2}.$$

10.2 Sobol and antithetics

Sobol points reduce integration error by filling the unit cube more evenly than independent random points. The implementation rounds the number of paths to a power of two because Sobol nets are naturally balanced in blocks of 2^m.

Antithetic pairing uses the fact that, for a standard normal vector Z,

$$Z \overset{d}{=} -Z.$$

The paired payoff

$$\frac{f(Z) + f(-Z)}{2}$$

has the same expectation as $f(Z)$. It often has lower variance for monotone or nearly monotone payoffs.

10.3 Default simulation policy

The default policy used by the project is:

1. Use Sobol low-discrepancy points whenever the path dimension is known in advance.
2. Use antithetic pairing when the payoff simulator accepts both Z and $-Z$.
3. Use a likelihood-ratio shift or shift mixture for tail-sensitive labels whenever a useful direction is available.
4. Use the unshifted component as part of the mixture unless a closed-form variance calculation says a pure shift is safe.
5. Use independent randomization or independent Sobol scrambles for benchmarks.

The phrase “whenever possible” matters. For example, if a payoff is synthetic and the useful tail direction is unknown, the safe choice is a mixture containing negative, zero, and positive shifts. If a benchmark uses a finite-difference or tree method, no MC importance sampling is needed because there is no path simulation in that benchmark.

10.4 From Sobol uniforms to GBM paths

For a d-asset GBM model over one step Δt, start with a Sobol vector

$$U = (U_1, \ldots, U_d) \in (0, 1)^d.$$

Map it to independent normals by

$$Z_i = \Phi^{-1}(U_i).$$

If L is a Cholesky factor of the correlation matrix ρ, so that $LL^\top = \rho$, set

$$\widetilde{Z} = LZ.$$

Then the simulated next spot is

$$S^{(j)}_{t+\Delta t} = S^{(j)}_t \exp\Big((r - q_j - \tfrac{1}{2}\sigma_j^2)\Delta t + \sigma_j \sqrt{\Delta t}\, \widetilde{Z}_j\Big).$$

For a path with many dates, repeat this step date by date. The Brownian increments are independent across time before correlation is applied across assets.

10.5 Likelihood-ratio importance sampling

State enrichment and importance sampling are different.

State enrichment changes the distribution of training states x_i. It does not change the path law for any individual conditional expectation.

Importance sampling changes the path density from p to q and multiplies by a likelihood ratio:

$$\mathbb{E}_p[h(Z)] = \mathbb{E}_q\left[h(Z)\frac{p(Z)}{q(Z)}\right].$$

For a normal mean shift, $q(z) = \phi(z - \theta)$, so

$$\frac{p(z)}{q(z)} = \exp\left(-\theta z + \frac{1}{2}\theta^2\right).$$

This identity is exact; its usefulness depends on variance.

10.6 Mixture importance sampling

A single shift can be too opinionated. The more generic tail-biased method is a mixture:

$$q(z) = \sum_{m=1}^{M} \pi_m \phi_d(z - \theta_m), \qquad \pi_m > 0, \quad \sum_{m=1}^{M} \pi_m = 1.$$

Here ϕ_d is the d-dimensional standard normal density and θ_m is a vector shift. The likelihood ratio is

$$L(z) = \frac{\phi_d(z)}{\sum_{m=1}^{M} \pi_m \phi_d(z - \theta_m)}.$$

The weighted estimator is

$$\widehat{V}_{\mathrm{IS}} = \frac{1}{N}\sum_{n=1}^{N} h(Z_n)L(Z_n), \qquad Z_n \sim q.$$

For option training, mixture shifts are useful because one model can include lower-tail, upper-tail, and central scenarios without needing a different formula for every product. The cost is that likelihood weights can become variable; therefore the code tracks benchmark error and avoids using an aggressive shift as the only source of paths.

10.7 Control variates

If C has known mean m_C, then

$$Y_\beta = Y - \beta(C - m_C)$$

has the same expectation as Y. The variance-minimizing coefficient is

$$\beta^\star = \frac{\mathrm{Cov}(Y, C)}{\mathrm{Var}(C)}.$$

Asian options use a geometric Asian control when the arithmetic payoff and geometric payoff are strongly correlated.

Chapter 11

Proxy Fitting Principles

11.1 Targets

Raw prices can be hard to fit because they range from nearly zero to almost linear. The code uses transformations such as:

$$\log(V + \varepsilon), \qquad \log \frac{V + \varepsilon}{B + \varepsilon}, \qquad \operatorname{logit} \frac{V - L}{U - L}.$$

Here B is a baseline, and L, U are known lower and upper bounds.

11.2 Exact regions

Never ask a regressor to learn something known analytically.

Examples:

- At maturity, use the payoff exactly.
- Deep Asian call in-the-money region: the positive part disappears and the value is linear.
- Cliquet global floor/cap tails: the final clipped payoff is already known.
- American/Bermudan exercise: impose $V = \max(\text{intrinsic}, \text{continuation})$.

11.3 Relative error floor

The reported relative error is

$$e_{\text{rel}} = \frac{|\widehat{V} - V|}{\max(|V|, \eta)}.$$

This is ordinary relative error when $|V| \geq \eta$, and it avoids meaningless exploding percentages when V is tiny.

Chapter 12

One-Dimensional Shape-Preserving Proxies

12.1 Why PCHIP is the default for 1D

For a single feature, PCHIP is a strong operational default:

- it interpolates labels exactly;
- it is local, so one noisy point does not create global oscillation;
- it preserves monotone shape when the data are monotone;
- it behaves well near exercise kinks, barrier transitions, and low-value tails.

12.2 Cubic Hermite form

On $[x_i, x_{i+1}]$, let $h_i = x_{i+1} - x_i$, $t = (x - x_i)/h_i$. A cubic Hermite segment is

$$\begin{aligned} H_i(t) = {} & y_i(2t^3 - 3t^2 + 1) + h_i d_i(t^3 - 2t^2 + t) \\ & + y_{i+1}(-2t^3 + 3t^2) + h_i d_{i+1}(t^3 - t^2). \end{aligned}$$

PCHIP chooses slopes d_i from neighboring secants in a way that prevents artificial extrema.

12.3 PCHIP slope construction

Let

$$\delta_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$$

be the secant slope. At an interior knot x_i, PCHIP applies this rule.

- If $\delta_{i-1}\delta_i \le 0$, set $d_i = 0$. This prevents a spurious turn at a sign change.

- If δ_{i-1} and δ_i have the same sign, set

$$d_i = \frac{w_1 + w_2}{w_1/\delta_{i-1} + w_2/\delta_i}, \qquad w_1 = 2h_i + h_{i-1}, \quad w_2 = h_i + 2h_{i-1}.$$

The second formula is a weighted harmonic mean. A harmonic mean is pulled toward the smaller magnitude, which stops the interpolant from using a large slope to overshoot a flat part of the curve.

Endpoint slopes use one-sided information and are then clipped to respect the first interior secant. This is why SciPy's `PchipInterpolator` is preferred in the main code: it gives a mature, tested implementation of the Fritsch-Butland endpoint and interior rules.

12.4 Why exact interpolation is acceptable

PCHIP interpolates labels exactly, so it is not a smoother. That is acceptable only when MC noise is already controlled by Sobol paths, antithetics, control variates, and enough paths. If labels are visibly noisy, a smoothing spline or local regression can have lower average error. The project therefore uses PCHIP as the default one-feature method but still treats label construction as the first-order accuracy problem.

12.5 Akima, splines, and Bernstein

Akima interpolation is also local and robust to outliers. Natural cubic splines are smoother but can overshoot. Bernstein polynomials are shape-friendly on bounded intervals and were competitive for some barrier variants. For one-dimensional production defaults, the current rule is:

PCHIP first, compare Akima/Bernstein when the validation surface says so.

Chapter 13

Sparse Chebyshev and Higher Dimension

13.1 Chebyshev scaling

Chebyshev polynomials satisfy

$$T_n(\cos\theta) = \cos(n\theta),$$

so $|T_n(x)| \leq 1$ for $x \in [-1, 1]$. This is why features are scaled before fitting:

$$z_j = 2\frac{x_j - \ell_j}{u_j - \ell_j} - 1.$$

13.2 Ridge regression

Given a design matrix A, labels y, and penalty matrix Γ, ridge regression solves

$$\min_{\beta} \|A\beta - y\|_2^2 + \lambda\|\Gamma\beta\|_2^2.$$

The normal equation is

$$(A^\top A + \lambda\Gamma^\top\Gamma)\beta = A^\top y.$$

13.3 Sparse multi-index basis

For a D-dimensional state $z = (z_1, \ldots, z_D) \in [-1, 1]^D$, a tensor Chebyshev basis term is

$$\Phi_\alpha(z) = \prod_{j=1}^{D} T_{\alpha_j}(z_j), \qquad \alpha = (\alpha_1, \ldots, \alpha_D).$$

A full tensor basis with $\alpha_j \leq p$ is usually too large. A total-degree basis keeps only

$$|\alpha|_1 = \alpha_1 + \cdots + \alpha_D \leq p.$$

The implementation also allows payoff-aware pruning. For example, a basket Asian correction keeps level, dispersion, running-sum, and a few PCA interaction terms before it considers high-order cross-products. This is not a pricing theorem; it is a bias-variance decision that must be checked by validation.

13.4 Baseline plus residual

High-dimensional products are easier to fit when the proxy is not asked to learn the whole price from scratch. The generic decomposition is

$$V(x) = B(x)\,R(x),$$

or, with an additive residual,

$$V(x) = B(x) + \varepsilon_{\mathrm{res}}(x).$$

Here $B(x)$ is a cheap baseline: Black-Scholes, moment-matched lognormal, intrinsic value plus discounted floor, or a low-dimensional approximation. The fitted target can then be

$$y(x) = \log \frac{V(x) + \epsilon}{B(x) + \epsilon}.$$

When B captures most of the shape, the residual is flatter and lower dimensional. This is the main reason the basket Asian and basket cliquet proxies improved after adding moment, PCA, and grouped-label features.

13.5 Why sparsity is necessary

A full tensor degree-p basis in D variables contains $(p+1)^D$ terms. With $D = 7$ and $p = 5$, that is 279,936 terms before fitting. Basket cliquet therefore needs sparse terms, payoff-aware features, or a slower safety method.

Chapter 14

Instrument Recipes

14.1 Recipe template

Each product section answers the same six questions:

1. What is the payoff?
2. What state is sufficient for future cash flows?
3. Which coordinate is used by the proxy?
4. How are MC labels generated?
5. What benchmark is used?
6. Which parts are exact identities and which parts are fitted approximations?

This template is important because a proxy can look accurate for the wrong reason. For example, reusing training paths in the benchmark can hide error, and using a coordinate that is not sufficient can look fine on a small grid but fail when parameters change.

14.2 European option

State: S. Coordinate:

$$d_1 = \frac{\log(S/K) + (r - q + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}.$$

Training labels use shifted Sobol terminal GBM paths. Validation uses Black-Scholes:

$$C = Se^{-q\tau}\Phi(d_1) - Ke^{-r\tau}\Phi(d_2), \qquad d_2 = d_1 - \sigma\sqrt{\tau}.$$

Default proxy: log price fitted by PCHIP in d_1.

Exact parts: the terminal GBM formula and the Black-Scholes benchmark. Fitted part: the interpolation from finitely many noisy labels. Main failure mode: tiny option values can make raw percentage error unstable, so validation reports both signed price error and relative error with a floor.

14.3 Arithmetic Asian option

State: S and running sum A. With m future fixings and total N,

$$K_{\mathrm{adj}} = \frac{NK - A - S}{m}.$$

The call payoff can be rewritten as

$$\left(\frac{A + S + \sum_{j=1}^{m} SG_j}{N} - K\right)_+ = S\frac{m}{N}\left(\frac{1}{m}\sum_{j=1}^{m} G_j - \frac{K_{\mathrm{adj}}}{S}\right)_+ .$$

Thus the tested monthly call can be represented by an adjusted-moneyness coordinate. Training uses Sobol antithetics, likelihood shifts where useful, and a geometric Asian control variate.

Exact parts: the payoff rewrite and deterministic in-the-money or out-of-the-money regions. Unbiased parts: Sobol randomized labels, antithetics, and likelihood weights. Approximate parts: using a one-dimensional adjusted-moneyness fit for the tested family and using MC for the arithmetic-average benchmark.

14.4 Ten-asset basket Asian

State: ten spots plus the running basket sum. The proxy starts from a moment-matched lognormal baseline for the final arithmetic basket average, then fits a sparse Chebyshev/PCA correction and a PCHIP residual calibration. PCA summarizes cross-sectional composition without throwing away the level and running-sum information.

Exact parts: first and second basket moments under GBM. Approximate parts: the lognormal moment match and the sparse residual proxy. The moment match is not expected to be exact because an arithmetic basket average is a sum of correlated lognormal variables, and that sum is not lognormal.

14.5 American and Bermudan options

At each exercise date k,

$$V_k(S) = \max\bigl(g_k(S),\, e^{-r\Delta t}\mathbb{E}[V_{k+1}(S_{k+1}) \mid S_k = S]\bigr) .$$

The continuation value is estimated by one-step Sobol transitions and fitted with PCHIP. The max with intrinsic value is imposed exactly.

Exact parts: dynamic-programming recursion and the exercise max. Approximate parts: finite exercise grid for Bermudan/American discretization, MC continuation labels, and interpolation of continuation. Benchmarks use a tree or projected finite-difference method to avoid testing the proxy against itself.

14.6 Barrier option

For a zero-rebate barrier,

$$V_{\text{in}} + V_{\text{out}} = V_{\text{vanilla}}.$$

Discrete monitoring checks simulated observation dates. Continuous monitoring uses Brownian-bridge survival between observations. For a lower log barrier b, endpoints $x, y > b$, and variance $\sigma^2 \Delta t$,

$$P(\text{survive} \mid x, y) = 1 - \exp\left(-\frac{2(x-b)(y-b)}{\sigma^2 \Delta t} \right).$$

Exact parts: discrete barrier path payoff and in/out parity. Conditional part: the Brownian-bridge survival probability is exact for continuous monitoring under log-GBM between two fixed endpoints. Approximate parts: MC integration and interpolation across spot.

14.7 Cliquet and SLV cliquet

Cliquet payoffs are bounded by local and global caps/floors. If accrued value a and m remaining coupons imply the final total is certainly below the global floor or above the global cap, the value is exact.

For single-name SLV cliquet, the state is (a, S, v). The proxy uses exact tails, bounded-logit targets, and local/sparse Chebyshev features. For three-asset basket cliquet, the fitted proxy is not universally enough for strict max-error control; hard order-statistic cases use a cached Sobol/LR safety proxy.

Exact parts: local coupon clipping, global floor/cap clipping, and tail bounds. Approximate parts: SLV time stepping, sparse fitted surfaces, and any cached MC safety proxy. The safety proxy is slower than a closed-form fitted proxy, but it is still much faster than regenerating a full benchmark for every repeated query.

14.8 Autocallable and random payoff

Autocallable valuation is naturally date-indexed. The state is spot at an observation date; the cash-flow rule determines whether the note redeems early or continues. Random payoff tests are synthetic one-dimensional payoffs used to stress interpolation under different shapes.

Exact parts: the call/no-call cash-flow rule at an observation date. Approximate parts: continuation labels and interpolation. The synthetic random payoff tests are not financial products; they are diagnostic shapes used to catch overshoot, poor extrapolation, and kink handling.

Chapter 15

Contract Definitions and Payoff Algebra

15.1 European call and put

For a call and put with strike K,

$$H_{\text{call}} = (S_T - K)_+, \qquad H_{\text{put}} = (K - S_T)_+.$$

The proxy problem is one-dimensional because, under constant-parameter GBM, all future uncertainty depends on the current state only through the current spot S_t. In practice the coordinate d_1, rather than raw spot, is preferred because equal spacing in d_1 allocates nodes by option delta rather than by currency units of spot.

15.2 Arithmetic Asian call

Let $0 = t_0 < t_1 < \cdots < t_{N-1} = T$ be fixing dates. At valuation date t_i, let

$$A_i = \sum_{j=0}^{i-1} S_{t_j}$$

be the running sum before today's fixing. If today's spot is S, then the final arithmetic average is

$$\bar{S}_T = \frac{A_i + S + \sum_{k=i+1}^{N-1} S_{t_k}}{N}.$$

The payoff is

$$H = (\bar{S}_T - K)_+.$$

The state is two-dimensional before reduction: $X_i = (S, A_i)$. The adjusted-strike algebra in Appendix C explains why the tested call can be fit on a one-dimensional adjusted-moneyness coordinate.

15.3 Basket Asian call

For d assets with basket weights w_j,

$$B_{t_k} = \sum_{j=1}^{d} w_j S_{t_k}^{(j)}, \qquad \bar{B}_T = \frac{R_i + B_{t_i} + \sum_{k=i+1}^{N-1} B_{t_k}}{N},$$

where R_i is the running sum of past basket fixings. The payoff is

$$H = (\bar{B}_T - K)_+.$$

This is no longer reducible to one scalar in general because the future basket distribution depends on the full cross-section of spots and correlations. The code therefore uses moment baselines and PCA features.

15.4 Barrier call

For a knock-out call with lower barrier L and upper barrier U, the alive indicator is

$$I_T = \mathbf{1}\{L < S_{t_k} < U \text{ for all monitored } t_k\}.$$

The payoff is

$$H_{\text{out}} = I_T (S_T - K)_+.$$

For a knock-in, $H_{\text{in}} = (1 - I_T)(S_T - K)_+$, so $H_{\text{in}} + H_{\text{out}} = H_{\text{vanilla}}$ path by path.

15.5 Cliquet payoff

For one underlying, the local return over period k is

$$R_k = \frac{S_{t_k}}{S_{t_{k-1}}} - 1.$$

The locally clipped coupon is

$$C_k = \text{clip}(R_k, \ell, u).$$

If $a_i = \sum_{k=1}^{i} C_k$ is the accrued clipped return at reset i, the final payoff is

$$H = N_0 \,\text{clip}\left(a_i + \sum_{k=i+1}^{M} C_k,\ G_L,\ G_U \right),$$

where N_0 is notional and G_L, G_U are global floor and cap.

15.6 Basket cliquet coupon variants

Let $R_k^{(j)}$ be the j-th asset return in period k. The repository tests several coupon maps. Representative examples are

$$
\begin{aligned}
C_k^{\text{basket}} &= \operatorname{clip}\left(\sum_j w_j R_k^{(j)}, \ell, u\right), \\
C_k^{\text{worst}} &= \operatorname{clip}\left(\min_j R_k^{(j)}, \ell, u\right), \\
C_k^{\text{best}} &= \operatorname{clip}\left(\max_j R_k^{(j)}, \ell, u\right), \\
C_k^{\text{spread}} &= \operatorname{clip}\left(\sum_j w_j R_k^{(j)}, \ell, u\right) - \lambda_s \operatorname{clip}\left(\max_j R_k^{(j)} - \min_j R_k^{(j)}, s_L, s_U\right) + b\,\mathbf{1}_{\{\sum_j w_j R_k^{(j)} \geq 0\}}.
\end{aligned}
$$

Order-statistic coupons are hard because small changes in cross-sectional dispersion can change which asset is best or worst.

15.7 Autocallable payoff recursion

At observation date k, if the note has not called and $S_{t_k} \geq B_{\text{call}}K$, the contract redeems. A schematic recursion is

$$
V_k(S) = \begin{cases} N_0(1 + c_k), & S \geq B_{\text{call}}K, \\ e^{-r\Delta t}\mathbb{E}[V_{k+1}(S_{k+1}) \mid S_k = S], & S < B_{\text{call}}K. \end{cases}
$$

At maturity the redemption rule may include coupon, capital protection, or downside participation.

Chapter 16

Implementation Algorithms

16.1 Generic proxy construction algorithm

For every instrument, the implementation follows this structure.

1. Define the contract payoff and observation schedule.
2. Prove or choose a Markov state X_t.
3. Transform the state to dimensionless coordinates: moneyness, d_1, normalized accrued return, normalized variance, PCA scores, or expected-total cushion.
4. Generate a training state grid. Include broad coverage and extra nodes near exercise, barrier, floor/cap, and low-value transition regions.
5. Generate Monte Carlo labels with Sobol, antithetics, control variates, and likelihood-ratio shifts where they lower variance.
6. Fit a proxy target such as $\log(V + \varepsilon)$, a bounded logit, or a baseline residual.
7. Validate on independent states and independent random streams.
8. Report values, signed errors, absolute errors, relative errors with a floor, and benchmark standard errors when available.

16.2 European algorithm

1. Fix $\tau = T - t$.
2. Choose training nodes $S_i = S(d_{1,i})$ with $d_{1,i}$ covering low and high deltas.
3. For each node, estimate V_i by shifted Sobol terminal paths:

$$S_T^{(i,n)} = S_i \exp\Big((r - q - \tfrac{1}{2}\sigma^2)\tau + \sigma\sqrt{\tau}\, Z_{i,n}^{\text{shifted}}\Big).$$

4. Fit $y_i = \log(V_i + \varepsilon)$ with PCHIP in d_1.
5. Predict by $\widehat{V}(S) = \exp(\widehat{y}(d_1(S))) - \varepsilon$, clipped at zero.
6. Benchmark by Black-Scholes.

16.3 Asian algorithm

1. For each fixing date, build states (S, A) in adjusted moneyness and running-average directions.
2. Compute $K_{\mathrm{adj}} = (NK - A - S)/m$.
3. Use exact linear/zero regions when $K_{\mathrm{adj}} \le 0$ or the option is certainly out of the money.
4. For the active region, simulate future fixings by Sobol antithetic GBM paths.
5. Estimate the arithmetic payoff and geometric payoff on the same paths.
6. Use the exact geometric Asian value as a control variate.
7. Fit normalized value and time value with PCHIP.
8. Validate with an independent high-path benchmark.

16.4 Basket Asian algorithm

1. Build states $X = (S^{(1)}, \ldots, S^{(10)}, R)$.
2. Compute the mean and variance of the final arithmetic basket average.
3. Price the moment-matched lognormal baseline $B(X)$.
4. Compute PCA scores of centered log spots to summarize composition.
5. Fit a sparse Chebyshev correction to $\log((V + \varepsilon)/(B + \varepsilon))$.
6. Bin residuals against expected-average moneyness and fit a PCHIP residual calibration.
7. Validate against Sobol/LR benchmarks on independent state grids.

16.5 American and Bermudan algorithm

1. Set terminal value equal to payoff.
2. Move backward through exercise dates.
3. At each date, simulate one-step transitions from training spots.
4. Evaluate the already-fitted next-date value on simulated next spots.
5. Average and discount to estimate continuation.

6. Fit continuation by PCHIP.
7. Define $V_k(S) = \max(g_k(S), \widehat{C}_k(S))$.
8. Validate American values against finite differences and Bermudan values against a tree.

16.6 Barrier algorithm

1. Build an alive-state spot grid inside the barrier domain.
2. Simulate discrete monitoring paths with Sobol antithetics.
3. For continuous monitoring, multiply each path by Brownian-bridge survival probabilities on each segment.
4. Fit the alive knock-out value by the best validated one-dimensional interpolator.
5. Obtain knock-in values by in/out parity when needed.
6. Validate by independent high-path simulation.

16.7 Cliquet and basket cliquet algorithm

1. Represent the state by accrued return and any market variables needed for the next returns.
2. Use exact global floor/cap tails before simulation.
3. Simulate remaining local returns under GBM or SLV.
4. Reuse each simulated future coupon sum for several accrued values when possible.
5. Fit a bounded target in logit space.
6. For high-dimensional order-statistic basket coupons, fall back to cached Sobol/LR safety pricing when fitted surfaces fail max-error targets.

Chapter 17

Validation and Timing

17.1 Independent benchmark principle

Training labels and benchmark labels use separate random streams. Benchmarks use more paths, closed forms, finite differences, or tree methods when available.

17.2 Current reporting

The timing report separates:

- training sample generation;
- proxy fitting after labels are available;
- option-family wall time;
- average wall time per scenario.

The reduced-budget smoke accuracy columns in the timing report are diagnostic only. The product-specific markdown summaries remain the full-budget accuracy source.

Appendix A

Line-by-Line Proofs: Conditional Pricing and State Choice

A.1 Risk-neutral pricing formula

Theorem A.1 (Risk-neutral valuation, constant short rate)**.** If the discounted asset gains are martingales under $\mathbb{Q}$ and the payoff H is integrable, then the time-t model value is

$$V_t = e^{-r(T-t)}\mathbb{E}_{\mathbb{Q}}[H \mid \mathcal{F}_t].$$

Line 1. The money-market account is $B_t = e^{rt}$.

Line 2. Absence of arbitrage in the risk-neutral model says the discounted value process $B_t^{-1}V_t$ is a $\mathbb{Q}$-martingale.

Line 3. Therefore

$$B_t^{-1}V_t = \mathbb{E}_{\mathbb{Q}}[B_T^{-1}H \mid \mathcal{F}_t].$$

Line 4. Since $B_t = e^{rt}$ and $B_T = e^{rT}$,

$$e^{-rt}V_t = e^{-rT}\mathbb{E}_{\mathbb{Q}}[H \mid \mathcal{F}_t].$$

Line 5. Multiply both sides by e^{rt} to obtain the formula.

A.2 When a proxy state is sufficient

Theorem A.2 (State reduction by conditional distribution)**.** Suppose X_t is $\mathcal{F}_t$-measurable and the conditional distribution of future cash flows H given $\mathcal{F}_t$ depends on the past only through X_t. Then there exists a function v such that

$$V_t = v(t, X_t).$$

Line 1. Define the price by conditional expectation:

$$V_t = e^{-r(T-t)}\mathbb{E}[H \mid \mathcal{F}_t].$$

Line 2. By assumption, once X_t is known, earlier path details do not change the conditional law of H.

Line 3. Therefore $\mathbb{E}[H \mid \mathcal{F}_t]$ is a function of X_t.

Line 4. Call this function $g(t, X_t)$.

Line 5. Set $v(t, X_t) = e^{-r(T-t)} g(t, X_t)$.

Line 6. Hence $V_t = v(t, X_t)$.

A.3 Tower property for backward induction

Theorem A.3 (Tower property)**.** If $\mathcal{G} \subseteq \mathcal{F}$, then

$$\mathbb{E}[\mathbb{E}[Y \mid \mathcal{F}] \mid \mathcal{G}] = \mathbb{E}[Y \mid \mathcal{G}].$$

Line 1. The conditional expectation $\mathbb{E}[Y \mid \mathcal{F}]$ is the best $\mathcal{F}$-measurable summary of Y in the sense of matching integrals over $\mathcal{F}$-events.

Line 2. Every $\mathcal{G}$-event is also an $\mathcal{F}$-event because $\mathcal{G} \subseteq \mathcal{F}$.

Line 3. Therefore replacing Y by $\mathbb{E}[Y \mid \mathcal{F}]$ preserves all integrals over $\mathcal{G}$-events.

Line 4. The conditional expectation with respect to $\mathcal{G}$ is the unique $\mathcal{G}$-measurable variable with those integrals.

Line 5. Hence the identity follows.

Appendix B

Line-by-Line Proofs: Monte Carlo

B.1 Unbiased sample mean

Line 1. Define $Y = e^{-r\tau}H$ and $V = \mathbb{E}_{\mathbb{Q}}[Y]$.

Line 2. Define $\widehat{V}_N = N^{-1}\sum_{n=1}^{N} Y_n$.

Line 3. Apply linearity:

$$\mathbb{E}[\widehat{V}_N] = \mathbb{E}\left[\frac{1}{N}\sum_{n=1}^{N} Y_n\right] = \frac{1}{N}\sum_{n=1}^{N}\mathbb{E}[Y_n] = \frac{1}{N}\sum_{n=1}^{N} V = V.$$

Line 4. If paths are independent with variance σ_Y^2,

$$\mathrm{Var}(\widehat{V}_N) = \mathrm{Var}\left(\frac{1}{N}\sum_{n=1}^{N} Y_n\right) = \frac{1}{N^2}\sum_{n=1}^{N}\mathrm{Var}(Y_n) = \frac{\sigma_Y^2}{N}.$$

Line 5. Replace σ_Y^2 by the sample variance to estimate standard error.

B.2 Likelihood-ratio shift

Line 1. Start with $V = \int h(z)p(z)\,\mathrm{d}z$.

Line 2. Choose a proposal q with $q(z) > 0$ wherever $h(z)p(z) \neq 0$.

Line 3. Multiply and divide by q:

$$V = \int h(z)\frac{p(z)}{q(z)}q(z)\,\mathrm{d}z = \mathbb{E}_q[h(Z)L(Z)], \qquad L(Z) = \frac{p(Z)}{q(Z)}.$$

Line 4. For $p(z) = \phi(z)$, $q(z) = \phi(z-\theta)$,

$$L(z) = \frac{\exp(-z^2/2)}{\exp(-(z-\theta)^2/2)} = \exp\left(-\theta z + \frac{\theta^2}{2}\right).$$

Line 5. For independent shocks, likelihood ratios multiply:

$$L(z_1, \ldots, z_m) = \prod_{j=1}^{m} \exp\left(-\theta_j z_j + \frac{\theta_j^2}{2}\right).$$

B.3 Mixture likelihood ratio

Line 1. Let the original density be $p(z)$.

Line 2. Choose shifted proposal densities $q_m(z)$ and probabilities π_m with $\sum_m \pi_m = 1$.

Line 3. Define the mixture density

$$q(z) = \sum_{m=1}^{M} \pi_m q_m(z).$$

Line 4. Require $q(z) > 0$ wherever $h(z)p(z) \neq 0$.

Line 5. Multiply and divide by q:

$$\int h(z)p(z)\,\mathrm{d}z = \int h(z)\frac{p(z)}{q(z)}q(z)\,\mathrm{d}z.$$

Line 6. Therefore, if $Z \sim q$,

$$\mathbb{E}_p[h(Z)] = \mathbb{E}_q[h(Z)L(Z)], \qquad L(Z) = \frac{p(Z)}{q(Z)}.$$

Line 7. To simulate from q, first draw a component M with probability π_m, then draw $Z \sim q_M$.

Line 8. The likelihood ratio must use the full mixture denominator $q(z)$, not only the chosen component density $q_M(z)$, because the sampled value z could also have been generated by other components.

B.4 Antithetic estimator

Line 1. Standard normal symmetry gives $Z \overset{d}{=} -Z$.

Line 2. Define $A = (f(Z) + f(-Z))/2$.

Line 3. Then

$$\mathbb{E}[A] = \frac{\mathbb{E}[f(Z)] + \mathbb{E}[f(-Z)]}{2} = \frac{\mathbb{E}[f(Z)] + \mathbb{E}[f(Z)]}{2} = \mathbb{E}[f(Z)].$$

Line 4. The variance is

$$\mathrm{Var}(A) = \frac{1}{2}\,\mathrm{Var}(f(Z)) + \frac{1}{2}\,\mathrm{Cov}(f(Z), f(-Z)).$$

Line 5. The pairing helps whenever the covariance term is negative.

Appendix C

Line-by-Line Proofs: GBM and European Options

C.1 GBM solution

Line 1. Start from $\mathrm{d}S_t/S_t = (r-q)\,\mathrm{d}t + \sigma\,\mathrm{d}W_t$.

Line 2. Apply Ito's lemma to $f(S) = \log S$:

$$\mathrm{d}\log S_t = \frac{1}{S_t}\,\mathrm{d}S_t - \frac{1}{2}\frac{1}{S_t^2}(\,\mathrm{d}S_t)^2.$$

Line 3. Since $(\,\mathrm{d}S_t)^2 = \sigma^2 S_t^2\,\mathrm{d}t$,

$$\mathrm{d}\log S_t = (r - q - \tfrac{1}{2}\sigma^2)\,\mathrm{d}t + \sigma\,\mathrm{d}W_t.$$

Line 4. Integrate over $[t, T]$:

$$\log(S_T/S_t) = (r - q - \tfrac{1}{2}\sigma^2)\tau + \sigma(W_T - W_t).$$

Line 5. Use $W_T - W_t = \sqrt{\tau}Z$, $Z \sim N(0,1)$, and exponentiate.

C.2 Black-Scholes call

Line 1. Risk-neutral pricing:

$$C = e^{-r\tau}\mathbb{E}[(S_T - K)_+].$$

Line 2. Split the positive part:

$$\mathbb{E}[(S_T - K)_+] = \mathbb{E}[S_T \mathbf{1}_{\{S_T > K\}}] - K\mathbb{Q}(S_T > K).$$

Line 3. The event $S_T > K$ is $Z > -d_2$, so $\mathbb{Q}(S_T > K) = \Phi(d_2)$.

Line 4. Completing the square gives

$$\mathbb{E}[S_T \mathbf{1}_{\{S_T > K\}}] = Se^{(r-q)\tau}\Phi(d_1).$$

Line 5. Substitute and discount:

$$C = Se^{-q\tau}\Phi(d_1) - Ke^{-r\tau}\Phi(d_2).$$

Appendix D

Line-by-Line Proofs: Asian State Reduction

D.1 Adjusted moneyness

Line 1. At a valuation date, write future fixings as $SG_1, \ldots, SG_m$.

Line 2. The arithmetic average is

$$\bar{S} = \frac{A + S + S\sum_{j=1}^{m} G_j}{N}.$$

Line 3. Define

$$K_{\text{adj}} = \frac{NK - A - S}{m}.$$

Line 4. Rewrite the call payoff:

$$(\bar{S} - K)_+ = \frac{1}{N}\left(A + S + S\sum_{j=1}^{m} G_j - NK\right)_+ = \frac{m}{N}\left(S\frac{1}{m}\sum_{j=1}^{m} G_j - K_{\text{adj}}\right)_+.$$

Line 5. Factor out $S > 0$:

$$(\bar{S} - K)_+ = S\frac{m}{N}\left(\frac{1}{m}\sum_{j=1}^{m} G_j - \frac{K_{\text{adj}}}{S}\right)_+.$$

Line 6. Under GBM the growth factors G_j do not depend on the current spot level S.

Line 7. Therefore $V(S, A) = SF(K_{\text{adj}}/S, t)$, which justifies a one-dimensional adjusted-moneyness proxy for the tested call setup.

D.2 Geometric Asian control variate

For future fixing times $t_1, \ldots, t_m$, define the geometric average

$$G = \left(\prod_{j=1}^{m} S_{t_j} \right)^{1/m}.$$

Line 1. Take logs:

$$\log G = \frac{1}{m} \sum_{j=1}^{m} \log S_{t_j}.$$

Line 2. Under GBM, each $\log S_{t_j}$ is normal.

Line 3. A linear combination of jointly normal random variables is normal.

Line 4. Therefore $\log G \sim N(\mu_G, \sigma_G^2)$.

Line 5. The mean is

$$\mu_G = \log S_0 + \frac{1}{m} \sum_{j=1}^{m} (r - q - \tfrac{1}{2}\sigma^2) t_j.$$

Line 6. The variance is

$$\sigma_G^2 = \frac{\sigma^2}{m^2} \sum_{i=1}^{m} \sum_{j=1}^{m} \min(t_i, t_j),$$

because $\mathrm{Cov}(W_{t_i}, W_{t_j}) = \min(t_i, t_j)$.

Line 7. Since G is lognormal, the call value $\mathbb{E}[(G - K)_+]$ has the same algebraic form as Black-Scholes with log-mean μ_G and log-variance σ_G^2.

Line 8. This exact geometric Asian value can be used as the known mean m_C in the control-variate formula.

Appendix E

Line-by-Line Proofs: Barriers

E.1 In/out parity

Line 1. Split all paths into two disjoint events: hit and no hit.

Line 2. On hit paths, the knock-in pays the vanilla payoff and the knock-out pays zero.

Line 3. On no-hit paths, the knock-out pays the vanilla payoff and the knock-in pays zero.

Line 4. Therefore, path by path,

$$H_{\text{in}} + H_{\text{out}} = H_{\text{vanilla}}.$$

Line 5. Discount and take expectations:

$$V_{\text{in}} + V_{\text{out}} = V_{\text{vanilla}}.$$

E.2 Brownian-bridge lower-barrier survival

Theorem E.1 (Brownian-bridge crossing probability)**.** Let X_s be a Brownian bridge from x to y over length Δt, with diffusion variance σ^2. If $x > b$ and $y > b$, then

$$\mathbb{Q}\left(\min_{0 \le s \le \Delta t} X_s \le b \,\middle|\, X_0 = x, X_{\Delta t} = y \right) = \exp\left(-\frac{2(x-b)(y-b)}{\sigma^2 \Delta t} \right).$$

Line 1. Condition on log endpoints $X_0 = x$, $X_{\Delta t} = y$, both above lower barrier b.

Line 2. By the reflection principle, the conditional probability of crossing below b is

$$\exp\left(-\frac{2(x-b)(y-b)}{\sigma^2 \Delta t} \right).$$

Line 3. Survival is one minus crossing:

$$P_{\text{surv}} = 1 - \exp\left(-\frac{2(x-b)(y-b)}{\sigma^2 \Delta t} \right).$$

Line 4. Conditional survival over many intervals is the product of interval survival probabilities.

Appendix F

Line-by-Line Proofs: Exercise and Cliquet Bounds

F.1 American recursion

Line 1. At final date M, $V_M(S) = g_M(S)$.

Line 2. Suppose V_{k+1} is known.

Line 3. Continuing from date k has value

$$C_k(S) = e^{-r\Delta t}\mathbb{E}[V_{k+1}(S_{k+1}) \mid S_k = S].$$

Line 4. Stopping immediately has value $g_k(S)$.

Line 5. The holder chooses the larger:

$$V_k(S) = \max(g_k(S), C_k(S)).$$

Line 6. Backward induction proves optimality at every date.

F.2 Cliquet exact tails

Line 1. Let current accrued return be a, with m future local coupons.

Line 2. Each future coupon lies in $[\ell, u]$.

Line 3. Therefore final accumulated return lies in

$$a + \sum_{j=1}^{m} c_j \in [a + m\ell,\ a + mu].$$

Line 4. If $a + mu$ is below the global floor, the payoff is exactly the global floor.

Line 5. If $a + m\ell$ is above the global cap, the payoff is exactly the global cap.

Line 6. These states need no regression and no simulation.

Appendix G

Line-by-Line Proofs: PCHIP

G.1 Unique Hermite cubic

Line 1. A cubic has four coefficients.

Line 2. Endpoint value constraints give two linear equations.

Line 3. Endpoint slope constraints give two more linear equations.

Line 4. The Hermite basis constructs one solution:

$$H(t) = y_i h_{00}(t) + h_i d_i h_{10}(t) + y_{i+1} h_{01}(t) + h_i d_{i+1} h_{11}(t).$$

Line 5. If two cubics satisfied all four conditions, their difference would have double roots at both endpoints.

Line 6. A nonzero cubic cannot have four roots counting multiplicity, so the solution is unique.

G.2 Why PCHIP avoids artificial extrema

Theorem G.1 (Fritsch-Carlson monotonicity criterion)**.** For a cubic Hermite interpolant on a monotone interval, if the normalized endpoint slopes satisfy the Fritsch-Carlson inequalities, then the derivative does not change sign inside the interval. Hence the cubic segment is monotone on that interval.

Line 1. Let $\delta_i = (y_{i+1} - y_i)/h_i$.

Line 2. If neighboring secants change sign, PCHIP sets the node slope to zero.

Line 3. If neighboring secants have the same sign, PCHIP uses the weighted harmonic mean

$$d_i = \frac{w_1 + w_2}{w_1/\delta_{i-1} + w_2/\delta_i}.$$

Line 4. This slope has the same sign as the neighboring secants and lies between their magnitudes.

Line 5. On a monotone interval, the derivative of the Hermite cubic is a quadratic whose normalized endpoint slopes satisfy the Fritsch-Carlson monotonicity inequalities.

Line 6. Therefore the derivative does not change sign inside the interval.

Line 7. Hence the interpolant cannot create a new local maximum or minimum between two monotone data points.

Appendix H

Line-by-Line Proofs: Sparse Chebyshev Ridge

H.1 Chebyshev boundedness

Line 1. Every $x \in [-1, 1]$ can be written as $x = \cos\theta$.

Line 2. Define $T_n(x) = \cos(n\theta)$.

Line 3. Since $|\cos(n\theta)| \leq 1$,

$$|T_n(x)| \leq 1, \qquad x \in [-1, 1].$$

Line 4. Scaling state variables to $[-1, 1]$ keeps basis columns numerically bounded.

H.2 Ridge normal equation

Line 1. Objective:

$$J(\beta) = \|A\beta - y\|_2^2 + \lambda\|\Gamma\beta\|_2^2.$$

Line 2. Differentiate:

$$\nabla J(\beta) = 2A^\top(A\beta - y) + 2\lambda\Gamma^\top\Gamma\beta.$$

Line 3. Set the gradient to zero:

$$A^\top A\beta - A^\top y + \lambda\Gamma^\top\Gamma\beta = 0.$$

Line 4. Rearrange:

$$(A^\top A + \lambda\Gamma^\top\Gamma)\beta = A^\top y.$$

Line 5. The left matrix is positive semidefinite, and ridge makes unsupported directions stable.

Appendix I

Line-by-Line Proofs: Bounded Targets and Metrics

I.1 Logit bounded target

Line 1. Suppose the price must lie in $[L, U]$.

Line 2. Fit an unconstrained number y and map it through the logistic function:

$$u(y) = \frac{1}{1 + e^{-y}}.$$

Line 3. Because $e^{-y} > 0$, $0 < u(y) < 1$.

Line 4. Rescale:

$$\widehat{V} = L + (U - L)u(y).$$

Line 5. Therefore $L < \widehat{V} < U$.

I.2 Relative-error floor

Line 1. Define

$$e_{\text{rel}} = \frac{|\widehat{V} - V|}{\max(|V|, \eta)}, \qquad \eta > 0.$$

Line 2. If $|V| \geq \eta$, this is ordinary relative error.

Line 3. If $|V| < \eta$, then

$$e_{\text{rel}} = \frac{|\widehat{V} - V|}{\eta}.$$

Line 4. Thus a nearly zero benchmark cannot create an infinite percentage error.

Appendix J

Line-by-Line Proofs: Basket Moments and PCA

J.1 Basket Asian first moment

Let the basket fixing at time t_k be

$$B_{t_k} = \sum_{j=1}^{d} w_j S_{t_k}^{(j)}.$$

Under correlated risk-neutral GBM,

$$\mathbb{E}[S_{t_k}^{(j)} \mid S_0^{(j)}] = S_0^{(j)} e^{(r-q_j)t_k}.$$

Therefore

$$\mathbb{E}[B_{t_k}] = \sum_{j=1}^{d} w_j S_0^{(j)} e^{(r-q_j)t_k}.$$

For the final arithmetic basket average

$$\bar{B} = \frac{R_i + B_{t_i} + \sum_{k=i+1}^{N-1} B_{t_k}}{N},$$

linearity gives

$$\mathbb{E}[\bar{B}] = \frac{R_i + B_{t_i} + \sum_{k=i+1}^{N-1} \mathbb{E}[B_{t_k}]}{N}.$$

J.2 Basket Asian second moment

For two assets j, ℓ and times t_a, t_b, correlated GBM gives

$$\mathbb{E}[S_{t_a}^{(j)} S_{t_b}^{(\ell)}] = S_0^{(j)} S_0^{(\ell)} e^{(r-q_j)t_a + (r-q_\ell)t_b + \rho_{j\ell}\sigma_j\sigma_\ell \min(t_a, t_b)}.$$

Then

$$\mathbb{E}[B_{t_a} B_{t_b}] = \sum_{j=1}^{d} \sum_{\ell=1}^{d} w_j w_\ell \mathbb{E}[S_{t_a}^{(j)} S_{t_b}^{(\ell)}].$$

Finally,

$$\mathrm{Var}(\bar{B}) = \mathbb{E}[\bar{B}^2] - \mathbb{E}[\bar{B}]^2,$$

where $\mathbb{E}[\bar{B}^2]$ is obtained by summing $\mathbb{E}[B_{t_a} B_{t_b}]$ over all future fixing pairs and adding deterministic running-sum terms.

J.3 Moment-matched lognormal baseline

Line 1. Suppose a positive average has mean m and variance v.

Line 2. Approximate it by $L = e^Y$, $Y \sim N(\mu_L, s_L^2)$.

Line 3. The lognormal mean and variance are

$$\mathbb{E}[L] = e^{\mu_L + s_L^2/2}, \qquad \mathrm{Var}(L) = e^{2\mu_L + s_L^2}(e^{s_L^2} - 1).$$

Line 4. Set $e^{\mu_L + s_L^2/2} = m$.

Line 5. Divide the variance equation by m^2:

$$\frac{v}{m^2} = e^{s_L^2} - 1.$$

Line 6. Hence

$$s_L^2 = \log(1 + v/m^2), \qquad \mu_L = \log m - \frac{1}{2} s_L^2.$$

Line 7. Use the Black-Scholes-style truncated-lognormal formula to value $(L - K)_+$.

J.4 PCA optimality

Theorem J.1 (Rayleigh-Ritz/PCA optimality)**.** For a centered random vector with covariance matrix Σ, the rank-k orthogonal projection minimizing mean-square reconstruction error projects onto the eigenvectors of Σ with the k largest eigenvalues.

Line 1. Let R be a centered random vector with covariance Σ.

Line 2. A rank-k orthogonal projection P has reconstruction error

$$\mathbb{E}[\|R - PR\|^2] = \mathrm{tr}(\Sigma) - \mathrm{tr}(P\Sigma).$$

Line 3. Minimizing error is therefore equivalent to maximizing $\mathrm{tr}(P\Sigma)$.

Line 4. The Rayleigh-Ritz theorem says this trace is maximized by projecting onto the eigenvectors with the k largest eigenvalues.

Line 5. Therefore PCA gives the best k-dimensional linear summary in mean-square reconstruction error.

Appendix K

Line-by-Line Proofs: Sobol, Design, and Validation

K.1 Why powers of two are used

Sobol sequences are digital nets in base two. The first 2^m points have the balance property for elementary binary boxes. If we stop at an arbitrary N, the last partial block may break this balance. The implementation therefore rounds a target path count to

$$N_{\text{Sobol}} = 2^{\lceil \log_2 N_{\text{target}} \rceil}.$$

This is not an unbiasedness theorem. It is a numerical integration design rule.

K.2 Common random numbers

Line 1. At state x_i, the label is $V(x_i) = \mathbb{E}[h(x_i, Z)]$.

Line 2. Using the same Z_n for several states gives

$$\widehat{V}(x_i) = \frac{1}{N} \sum_{n=1}^{N} h(x_i, Z_n).$$

Line 3. For each fixed i, the marginal distribution of Z_n is unchanged.

Line 4. Therefore each $\widehat{V}(x_i)$ remains unbiased.

Line 5. The errors across nearby states become correlated, which often makes label noise smoother and easier for interpolation to absorb.

K.3 Benchmark independence

If the same paths are used for training labels and benchmark labels, the measured proxy error can be artificially small because both contain the same random noise. Independent benchmark streams estimate

$$\widehat{V}_{\text{proxy}}(x) - V_{\text{model}}(x)$$

with less leakage. When a closed-form, finite-difference, or tree benchmark exists, it is preferred because it removes benchmark Monte Carlo noise.

Appendix L

Line-by-Line Proofs: SLV and Basket Cliquet

L.1 Variance process discretization

The SLV cliquet code uses an Euler-type variance update with positivity protection. A schematic Heston-style variance step is

$$v_{t+\Delta t} = \max\left(v_t + \kappa(\theta - v_t)\Delta t + \xi\sqrt{v_t^+}\sqrt{\Delta t}\, Z_v,\ 0\right).$$

The max is not an exact theorem; it is a numerical safeguard. The consequence is that benchmark quality must be checked by increasing time steps or comparing against a stronger scheme when precision matters.

L.2 Local volatility multiplier

A local volatility multiplier $L(S)$ changes the spot diffusion to

$$\frac{\mathrm{d}S_t}{S_t} = (r - q)\,\mathrm{d}t + L(S_t)\sqrt{v_t}\,\mathrm{d}W_t.$$

The pricing theorem still applies once the simulator defines the risk-neutral dynamics. The proxy must include enough state information, at least S_t and v_t, because the future law depends on both.

L.3 Grouped accrued labels

For cliquets, suppose a simulated future path gives a remaining coupon sum $C^{(n)}$. For accrued values $a_1, \ldots, a_L$,

$$H_\ell^{(n)} = N_0\,\mathrm{clip}(a_\ell + C^{(n)}, G_L, G_U).$$

One simulation of $C^{(n)}$ prices all L accrued layers. This is exact path reuse, not approximation, because the same future path sum is valid for every starting accrued value.

L.4 Why basket order statistics are hard

For a worst-of coupon,

$$C_k = \mathrm{clip}(\min_j R_k^{(j)}, \ell, u).$$

The identity of the minimum can change abruptly as the state changes. This creates nonsmooth interaction surfaces in spot and variance. A low-degree global polynomial can miss these switching surfaces. That is why the methodology allows a slower cached Sobol/LR safety proxy for strict max-error cases.

References

- Black, F. and Scholes, M. (1973). The Pricing of Options and Corporate Liabilities. *Journal of Political Economy.*

- Fritsch, F. N. and Carlson, R. E. (1980). Monotone Piecewise Cubic Interpolation. *SIAM Journal on Numerical Analysis.*

- Fritsch, F. N. and Butland, J. (1984). A Method for Constructing Local Monotone Piecewise Cubic Interpolants. *SIAM Journal on Scientific and Statistical Computing.*

- Hoerl, A. E. and Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics.*

- Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering.* Springer.

- Glasserman, P. and Staum, J. (2001). Conditioning on One-Step Survival for Barrier Option Simulations. *Operations Research.*

- Broadie, M., Glasserman, P., and Kou, S. G. (1997). A Continuity Correction for Discrete Barrier Options. *Mathematical Finance.*